qbank — módulo de generación de preguntas (parte export)

Marcos Bujosa

26 de octubre de 2026

Índice

1. Dando formato a las preguntas **2**
1.1. Preguntas en LaTeX para usar con Auto-Multiple-Choice 3
1.2. Escritura del código LaTeX para las preguntas en formato AMC 6
1.3. Preguntas en XML para usar con Moodle 9
1.4. Preguntas multi-parte (AMC) y cloze (Moodle) 23

2. Plantilla de proyecto: `qbank-init` **27**

1. Dando formato a las preguntas

Estructura del módulo **qbank._export**

```
# Copyright (C) 2020-2026  Marcos Bujosa
#
# This file is part of calcprop-qbank.
#
# calcprop-qbank is free software: you can redistribute it and/or modify
# it under the terms of the GNU General Public License as published by
# the Free Software Foundation, either version 3 of the License, or
# (at your option) any later version.
#
# calcprop-qbank is distributed in the hope that it will be useful,
# but WITHOUT ANY WARRANTY; without even the implied warranty of
# MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
# GNU General Public License for more details.
#
# You should have received a copy of the GNU General Public License
# along with this program.  If not, see <https://www.gnu.org/licenses/>.

import os as _os
import re as _re
from qbank._quiz import *
from qbank._quiz import _normaliza_slot, _clasifica_slot   # no los exporta el *

# ── Helpers ──────────────────────────────────────────────────────────────

<<Helper _sanitize_name>>
<<Método codchar para codificar caracteres no ASCII en LaTeX>>

# ── Exportadores AMC ─────────────────────────────────────────────────────

<<AMCblock>>


# ── AMC para ProblemaVF y multiparte ─────────────────────────────────────

<<Formato AMC Verdadero/Falso>>
<<AMC multi-parte>>

# ── Exportadores Moodle ──────────────────────────────────────────────────

<<_MOODLE_HEADER y _moodle_header>>
<<Quiz para Moodle>>
<<Quiz para Moodle versión Profe>>


<<Quiz AMC versión Profe>>
<<Quiz de lectura>>
<<Quiz VF para Moodle>>
<<Formato Moodle de preguntas de elección múltiple>>
<<Formato Moodle de preguntas de elección múltiple para el Profe>>
<<Formato Moodle de preguntas de elección múltiple con LastChoice>>


# ── Bloques Cloze (Moodle interno) ───────────────────────────────────────

<<Moodle cloze multi-parte>>
```

1.1. Preguntas en LaTeX para usar con Auto-Multiple-Choice

Para entender el comportamiento de las funciones de exportación, primero debemos conocer la estructura de una pregunta de opción múltiple en AMC.

Prestemos atención al siguiente ejemplo canónico extraído de la documentación oficial de Auto Multiple Choice (AMC):

```
\element{general}{
  \begin{questionmult}{pref}
    Entre las siguientes ciudades, ¿cuáles son prefecturas francesas?
    \begin{choices}
      \correctchoice{Poitiers}
      \wrongchoice{Sainte-Menehould}
      \correctchoice{Avignon}
    \end{choices}
  \end{questionmult}
}
```

Como se puede observar, cada pregunta se encapsula dentro del comando `\element{}{}`, el cual exige dos argumentos fundamentales:

Primer argumento: Una etiqueta alfanumérica que identifica el grupo temático o categoría de la pregunta (por ejemplo, `general`, o etiquetas técnicas como `TestSignificacionIndividual` o `Diagonalizacion`). Agrupar variantes bajo un mismo identificador temático permite a AMC confeccionar exámenes equilibrados seleccionando aleatoriamente un número determinado de ejercicios de cada bloque. Utilizaremos esta etiqueta como el nexo común para todas las variantes generadas por nuestras clases.

Segundo argumento: El bloque de la pregunta en sí. En nuestro sistema, este argumento se alimentará dinámicamente con la variante concreta obtenida al iterar `ProblemaTipo`, estructurada bajo el entorno `questionmult` de LaTeX.

Dentro del entorno `questionmult` intervienen tres componentes:

1. Una etiqueta identificadora única para la variante concreta (en nuestro caso, utilizaremos el índice secuencial provisto por el iterador).
2. El enunciado del ejercicio, compuesto por la concatenación de los textos (`self.e`) de los objetos `Supuesto` que se hayan combinado al iterar `ProblemaTipo`.
3. El entorno `choices`, que alberga las opciones a evaluar. Aquí se vuelcan los textos (`self.e`) de los objetos `Cuestion` seleccionados al iterar `ProblemaTipo`. Si la cuestión es lógicamente verdadera para la combinación de supuestos, se imprimirá como argumento del comando `\correctchoice{}`; si es falsa, se imprimirá como argumento del comando `\wrongchoice{}`.

El objetivo de este módulo es procesar las variantes generadas por los iteradores de Python y transformarlas de forma automatizada en cadenas de texto con código LaTeX válido para AMC.

1.1.1. Helpers: identificadores y codificación de caracteres

Función `_sanitize_name` Al construir un banco de preguntas es natural dar a cada grupo un nombre descriptivo que incluya el tema o el número de lección, como `L-07:Multicolinealidad` o `Tema3:Inferencia`. Estos nombres terminan incrustados en dos lugares problemáticos: el entorno `\element{nombre}{...}` de AMC y el entorno `\begin{cloze}{nombre-etiqueta}` de Moodle, y además en las reglas del `Makefile` que compila los ficheros. El carácter `:` es interpretado como delimitador de objetivo por `make` y como separador de etiqueta por algunos parsers LaTeX, con

resultados impredecibles: errores en la compilación, ficheros con nombres incorrectos o incluso silencio (el parser simplemente ignora parte del nombre).

Para que el usuario no tenga que preocuparse por estas restricciones —y pueda nombrar sus bancos con la notación que le resulte más natural—, todas las funciones de exportación llaman internamente a `_sanitize_name` antes de insertar el nombre en el código LaTeX. El usuario escribe `L-07:Multicolinealidad`; la función entrega al compilador `L-07-Multicolinealidad` (y sustituye igual el guion bajo, `_`, que LaTeX toma por subíndice):

```
def _sanitize_name(nombre):
    """Sustituye ':' y '_' por '-' en identificadores LaTeX."""
    return nombre.replace(':', '-').replace('_', '-')
```

Función `codchar` Para evitar la tediosa escritura manual de acentos y eñes al estilo LaTeX (como `\'{a}` o `\~{n}`), delegamos la tarea en Python mediante la función `codchar(s)`, que realiza una sustitución masiva de strings. El paquete `moodle` convierte esas órdenes en entidades HTML (`á`, `ü`, `¿`, etc.); un carácter que se le escape llega al XML en UTF-8 aunque el fichero se declare `iso-8859-1`, y Moodle lo muestra mal (`antigÃ¼edad`). Por eso cubre también las mayúsculas, la diéresis y los signos de apertura ¿ y ¡ (con `\textquestiondown` y `\textexclamdown` seguidos de un espacio, que TeX consume: unas llaves vacías llegarían literales al HTML):

```
def codchar(s):
     s = s.replace("á", "\\'{a}")
     s = s.replace("é", "\\'{e}")
     s = s.replace("í", "\\'{i}")
     s = s.replace("ó", "\\'{o}")
     s = s.replace("ú", "\\'{u}")
     s = s.replace("ñ", "\\~{n}")
     s = s.replace("ü", '\\"{u}')
     s = s.replace("Á", "\\'{A}")
     s = s.replace("É", "\\'{E}")
     s = s.replace("Í", "\\'{I}")
     s = s.replace("Ó", "\\'{O}")
     s = s.replace("Ú", "\\'{U}")
     s = s.replace("Ñ", "\\~{N}")
     s = s.replace("Ü", '\\"{U}')
     s = s.replace("¿", "\\textquestiondown ")
     s = s.replace("¡", "\\textexclamdown ")
     return s
```

1.1.2. Escritura de las cuestiones dentro del entorno `choices`

Texto de las cuestiones en función de si son verdaderas o falsas Recordemos que al ejecutar:

```
# 1. Instanciación del iterador
variantes = iter(ProblemaTipo(ejercicio))

# 2. Extracción y desempaquetado de una variante
id_pregunta, EnunciadoCompleto, lista_Cuestiones = next(variantes)
```

`id_pregunta` es un identificador único o índice para la variante generada (por ejemplo, `'3'`).

`EnunciadoCompleto` es el texto del enunciado principal de la variante (por ejemplo, `'Considere B y que B equivale a D. Conteste: '`).

lista_Cuestiones es una lista de tuplas, donde cada tupla representa una de las opciones o subcuestiones a evaluar para esta variante. Por ejemplo `[('¿Se da D?', True, 1, ''), ('¿Se dan C y D?', False, 1, '')]`

Cada tupla `c` de la `lista_Cuestiones` está estructurada de la siguiente forma:

```
('Texto de la opción/cuestión', True/False, 1/0, 'explicación si ha sido incluida')
```

Para escribir las cuestiones dentro del entorno de LaTeX `choices`, explotaremos la estructura de cada tupla `c`:

- Para asegurar la legibilidad del fichero LaTeX generado, el código formatea la salida aplicando una indentación fija de siete espacios.
- Se evalúa la veracidad en `c[1]` para seleccionar el comando adecuado
- Finalmente se cierra la llave correspondiente al comando `}` y se inserta un salto de línea (`\n`).

```
s = s + (' ' * 7) + ('\\correctchoice{' if c[1] else '\\wrongchoice  {') + c[0] + '}\n'
```

Última opción por defecto (`lastchoices`) AMC permite fijar opciones de respuesta al final de la lista mediante el comando `\lastchoices`, impidiendo que el motor de barajado las mezcle con las demás opciones. Esto es muy útil para introducir la clásica opción de escape cuando ninguna de las alternativas presentadas es lógicamente válida.

Examinemos este comportamiento en el siguiente fragmento de la documentación de AMC:

```
\begin{questionmult}{number}
  ¿Cuántos?
  \begin{choiceshoriz}
    \wrongchoice{ninguno}
    \correctchoice{uno}
    \wrongchoice{dos}
    \wrongchoice{tres}
    \lastchoices
    \correctchoice{no tanto}
    \wrongchoice{mucho}
  \end{choiceshoriz}
\end{questionmult}
```

En nuestras funciones, el argumento `last_choice_text` (cuyo valor predeterminado es `'Ninguna de las anteriores'`) se inyectará tras el delimitador `\lastchoices`. Su veracidad se evalúa dinámicamente: si al menos una de las cuestiones de la lista evaluada resultó ser verdadera (`True`), la opción por defecto se marcará automáticamente como `\wrongchoice`. Si todas las cuestiones de la variante fueron falsas, se convertirá en la opción `\correctchoice`. Solo cuentan las cuestiones activas (`c[2]` distinto de cero): en la vista profe una rechazada lleva en `c[1]` la cadena con el motivo, que como valor lógico es verdadera, y la opción por defecto saldría falsa aunque todas las activas lo fueran.

```
# (fragmento de _amc_choices)
if last_choice:
    s += ' ' * 7 + '\\lastchoices\n'
    s += ' ' * 7 + ('\\wrongchoice  {' if any(c[2] and c[1] for c in cuestiones) else
    ↪  '\\correctchoice{') + last_choice_text + '}\n'
```

1.1.3. Información sobre las cuestiones descartadas (para revisión del profesor)

AMC permite crear una copia del examen (o del banco completo de preguntas) con las respuestas correctas ya cumplimentadas en el PDF. Pensando en la posibilidad de distribuir estas copias a los alumnos, ofrece el entorno `\explain` para añadir explicaciones adicionales que les ayuden a comprender mejor el motivo por el que unas opciones son correctas y otras no. El texto contenido en este bloque queda oculto en los exámenes de los estudiantes, pero se compila y muestra de forma explícita en los documentos de tipo **Solución** o **Catálogo**.

Nosotros usaremos `\explain` con un propósito distinto: facilitar las tareas de auditoría y revisión de los bancos de preguntas. Cuando el tercer elemento de la tupla de la cuestión sea igual a cero (`c[2] == 0`), indicando que la pregunta ha sido descartada por el incumplimiento de una precondición, la función no la enviará al entorno de respuestas, sino que concatenará su texto y el motivo del descarte en una cadena global de diagnóstico (`ex`) que se volcará dentro del comando `\explain{}`.

```
# (fragmento de _amc_choices: c[2] es 0 para las rechazadas y c[1] el motivo)
elif profe:
    ex += ' cuestión: ' + texto + '; ' + str(c[1]) + '\n'
```

1.2. Escritura del código LaTeX para las preguntas en formato AMC

1.2.1. Función unificada `AMCblock`

Al desarrollar el módulo en sus primeras versiones, cada combinación de características de exportación daba lugar a una función independiente: `AMC`, `AMClastCh`, `AMCmc`, `AMClastChmc`, `AMCProfe`, `AMCmcProfe`… El problema es que la combinatoria crece rápidamente: basta añadir un nuevo parámetro para que el número de funciones necesarias se duplique. La solución es la habitual en diseño de APIs: colapsar toda la familia en una única función con parámetros opcionales que, por defecto, producen el comportamiento más común.

`AMCblock` es esa función unificada. Los tres ejes de variación que antes requerían funciones distintas se convierten ahora en argumentos de palabra clave con valores predeterminados:

- ¿Queremos opción por defecto al final? → `last_choice=True` (por defecto `False`).
- ¿Queremos las opciones distribuidas en varias columnas? → `cols=2` (por defecto `1`).
- ¿Queremos el diagnóstico de cuestiones descartadas por precondición? → `profe=True` (por defecto `False`).

Adicionalmente, `AMCblock` realiza dos transformaciones automáticas que antes había que gestionar a mano:

1. **Sanitización del nombre**: llama internamente a `_sanitize_name` para que los identificadores con `:` o `_` no provoquen errores en `make` ni en el compilador de LaTeX.
2. **Conversión de matemáticas en modo display**: los bloques `$$...$$` del enunciado se reescriben como `\[...\]`. AMC no reconoce el entorno `$$` de LaTeX y los compilaría sin añadir el salto de línea esperado; el entorno `\[...\]` es su equivalente correcto para matemáticas en bloque independiente.

Los parámetros que acepta son:

`nombre` Cadena que identifica la categoría del banco de ítems. Se sanitiza automáticamente (`:` y `_` → `-`) para compatibilidad con `make` y parsers LaTeX.

`etiqueta` Identificador único de la variante concreta (el índice secuencial del iterador).

enunciado El bloque de texto principal. Los bloques `$$...$$` se convierten automáticamente a `\[...\]`.

cuestiones Lista de tuplas `(texto, correcto, activa[, exp])`, tal como la produce el iterador de `ProblemaTipo`.

last_choice Si `True`, añade `\lastchoices` con la opción por defecto al final (ver sección anterior).

cols Número de columnas (>1 activa `\begin{multicols}{N}\AMCBoxedAnswers`).

profe Si `True`, añade `\explain` con las cuestiones descartadas por precondición (ver sección anterior).

aux_latex Instrucciones LaTeX adicionales que se insertan dentro del bloque `\element{}` (por ejemplo, un cambio de tamaño de letra). Solo por palabra clave.

last_choice_text Texto de la opción por defecto (por defecto `'Ninguna de las anteriores'`). Solo por palabra clave.

```
def AMCblock(nombre, etiqueta, enunciado, cuestiones,
             last_choice=False, cols=1, profe=False, *,
             aux_latex="", last_choice_text="Ninguna de las anteriores"):
    """Genera el bloque AMC para una pregunta de tipo multichoice.

    Parámetros
    ----------
    nombre           : identificador del grupo AMC (se sanitiza: ':' → '-')
    etiqueta         : etiqueta de la variante
    enunciado        : texto del enunciado (``$$...$$`` se convierte a ``\\[...\\]``)
    cuestiones       : lista de (texto, correcto, activa[, exp])
    last_choice      : si True, añade la opción por defecto con ``\\lastchoices``
    cols             : número de columnas multicols (1 = sin multicols)
    profe            : si True, añade ``\\explain`` con cuestiones rechazadas
    aux_latex        : instrucciones LaTeX extra dentro del bloque \\element{}
    last_choice_text : texto de la opción por defecto
    """
    nombre_tex = _sanitize_name(nombre)
    s  = '\\element{' + nombre_tex + '}{' + aux_latex + '\n'
    s += ' \\begin{questionmult}{' + nombre_tex + '-' + str(etiqueta) + '}\n'
    s += '  ' + _fix_display(enunciado) + '\n'
    choices, ex = _amc_choices(cuestiones, last_choice, cols, profe, last_choice_text)
    s += choices
    if profe and ex:
        s += '   \\explain{' + ex + '            }\n'
    s += ' \\end{questionmult} '
    s += '}\n\n'
    return s


def _fix_display(t):
    """``$$…$$`` → ``\\[…\\]`` (automultiplechoice se lleva mal con ``$$``)."""
    return _re.sub(r'\$\$(.+?)\$\$', r'\\[\1\\]', t, flags=_re.DOTALL)


def _amc_choices(cuestiones, last_choice, cols, profe, last_choice_text):
    """Bloque ``\\begin{choices}…\\end{choices}`` de una parte (dentro de
    multicols si cols > 1, con la opción por defecto si last_choice) y, para
    la vista profe, el texto de las cuestiones rechazadas que irá en ``\\explain``."""
    s, ex = '', ''
```

```
    if cols > 1:
        s += '   \\begin{multicols}{' + str(cols) + '}\\AMCBoxedAnswers\n'
        ch_indent = '      '
    else:
        ch_indent = '     '

    s += ch_indent + '\\begin{choices}\n'
    for c in cuestiones:
        texto = _fix_display(c[0])
        if c[2]:
            s += ' ' * 7 + ('\\correctchoice{' if c[1] else '\\wrongchoice  {') + texto + '}\n'
        elif profe:
            ex += ' cuestión: ' + texto + '; ' + str(c[1]) + '\n'

    if last_choice:
        s += ' ' * 7 + '\\lastchoices\n'
        s += ' ' * 7 + ('\\wrongchoice  {' if any(c[2] and c[1] for c in cuestiones) else
        ↪  '\\correctchoice{') + last_choice_text + '}\n'

    s += ch_indent + '\\end{choices}\n'

    if cols > 1:
        s += '   \\end{multicols}\n'
    return s, ex
```

`AMCblock` delega en dos auxiliares que comparte con `AMC_multipart`: `_fix_display`, que convierte `$$…$$` en `\[…\]`, y `_amc_choices`, que escribe el bloque de casillas de *una* parte (con `multicols`, la opción por defecto y, para la vista profe, el registro de las cuestiones rechazadas).

1.2.2. Variante para preguntas Verdadero/Falso en formato para exámenes en el aula

Las preguntas aleatorizadas generadas por la clase `ProblemaVF` poseen una estructura simplificada: carecen de precondiciones de filtrado dinámico. En consecuencia, el parámetro flag de descarte de la tupla (`c[2]`) no existe. Las tuplas inyectadas desde dicho banco constan únicamente de dos componentes: el texto y su booleano de veracidad: ‘(Texto, True/False)‘.

A continuación se define la función específica encargada de iterar directamente esta colección binaria para renderizar la salida de examen:

```
def AMC_VF(nombre, etiqueta, enunciado, cuestiones, opc=None):
    if opc is None:
        opc = ["", "Ninguna de las anteriores"]
    InstruccionesAux = opc[0]
    OpcPorDefecto    = opc[1]
    nombre = _sanitize_name(nombre)
    s = '\\element{' + nombre + '}{' + InstruccionesAux + '\n'
    s = s + ' \\begin{questionmult}{' + nombre + '-' + str(etiqueta) + '}\n'
    s = s + '  ' + enunciado + '\n'

    s = s + '     \\begin{choices}\n'
    for c in cuestiones:
        <<Escritura del texto de la cuestión en función de su veracidad>>
    s = s + '     \\end{choices}\n'

    s = s + ' \\end{questionmult} '
    s = s + '}\n\n'
    return s
```

Ejemplo de uso El siguiente bloque implementa el volcado físico del código generado por `AMC_VF` a un archivo con extensión `.tex`, creando un examen de 4 variantes compuestas por 3 preguntas de verdadero/falso seleccionadas aleatoriamente de nuestro repositorio básico.

```
from qbank import *

enunciado = "Indique qué afirmaciones son verdaderas:"
bancoVF = [
     ("Todo alumno que va a clase aprueba",   False),
     ("Todo alumno que suspende va a clase",  False),
     ("Todo alumno que sabe aprueba",         True ),
     ("Si aprueban todos eres buen profesor", False),
     ("Si suspenden todos eres mal profesor", False),
     ("Si suspenden todos es frustrante",     True ),]
GenVar = iter( ProblemaVF (enunciado, bancoVF, 3) ) # Tres preguntas por variante

import os
nombre     = "EjemploVF"
directorio = "ejemplos/"
os.makedirs(directorio, exist_ok=True)
with open(directorio + nombre + ".tex","w") as f:
    for i in range(4):                           # Cuatro variantes
        var = (next(GenVar))
        f.write(AMC_VF(nombre,var[0],var[1],var[2]))
```

1.2.3. Ejemplo de uso de `AMCblock`

A continuación se presenta un ejemplo completo de diseño de una pregunta combinatoria. El argumento `last_choice=True` añade automáticamente la opción por defecto al final y `cols=2` distribuye las respuestas en dos columnas:

```
from qbank import *
p = ProblemaTipo([
    "Considera ",
    [Supuesto("A ", v("A")), Supuesto("B ", v("B"))],
    [Supuesto("y A⇒C. ", v("A") >> v("C")),
     Supuesto("y B⟺D. ", v("B") ** v("D"))],
    "Indica qué es cierto: ",
    [Cuestion("C es verdadero", v("C")),
     Cuestion("D es verdadero", v("D"))],
], export={"last_choice": True, "cols": 2})

nombre = "MiPregunta"
with open(nombre + ".tex", "w") as f:
    for etiqueta, partes in p.por_partes():
        enunciado, cuestiones = partes[0]
        f.write(AMCblock(nombre, etiqueta, enunciado, cuestiones,
                         last_choice=True, cols=2))
```

El campo `export` del `ProblemaTipo` permite guardar la configuración de exportación junto al problema en JSON (`last_choice`, `cols`), de modo que los scripts de generación puedan leerlo sin hardcodear los parámetros.

1.3. Preguntas en XML para usar con Moodle

Para generar los ficheros `xml` necesarios para Moodle, seguiremos un método indirecto. Escribir directamente en XML es tedioso debido a la gestión de caracteres no ASCII y las expresiones

matemáticas de LaTeX.

La solución más práctica consiste en generar desde Python un fichero `.tex` de LaTeX que emplee el paquete moodle. Al compilar dicho fichero (con `pdflatex`, `lualatex` o `xelatex`) obtendremos simultáneamente una versión en PDF del banco de preguntas (ideal para revisión) y el archivo `.xml` listo para ser importado en Moodle. Además, este enfoque nos evita lidiar con la compleja estructura del XML de Moodle, sustituyéndola por la familiar sintaxis de LaTeX.

1.3.1. Preámbulo de los ficheros LaTeX para exportación a Moodle

Para que se genere el archivo XML, es estrictamente necesario que el preámbulo del fichero `.tex` cargue el paquete `moodle`.

El preámbulo definido a continuación está configurado para que el documento PDF resultante tenga un tamaño de fuente de `11pt` y márgenes estrechos mediante el paquete `fullpage` para optimizar la lectura del docente. Asimismo, se cargan `graphicx` y `fancyvrb` para soportar gráficos y código literal (todo esto es opcional).

La plantilla se almacena en la constante `_MOODLE_HEADER` y la función `_moodle_header` la instancia con el nombre del quiz y el preámbulo LaTeX opcional:

```
_MOODLE_HEADER = """\
\\documentclass[11pt]{{article}}

\\usepackage[cm,headings]{{fullpage}}

\\usepackage{{moodle}}

\\usepackage{{graphicx}}

\\usepackage{{fancyvrb}}

\\ifPDFTeX                    % FOR LATEX and PDFLATEX
    \\usepackage[utf8]{{inputenc}}   % necessary
    \\usepackage[OT1] {{fontenc}}    % necessary
\\else                        % assuming XELATEX or LUALATEX
    \\usepackage{{fontspec}}
\\fi

{auxLaTeX}
\\newcommand\\peque{{}}

\\begin{{document}}

\\begin{{quiz}}{{{nombre_tex}}}

"""

def _moodle_header(nombre, auxLaTeX=""):
    return _MOODLE_HEADER.format(
        auxLaTeX=auxLaTeX,
        nombre_tex=_sanitize_name(nombre),
    )
```

1.3.2. Estructura de preguntas de opción múltiple con el paquete moodle

La sintaxis nativa de `moodle.sty` para una pregunta de opción múltiple con una única respuesta correcta sigue la siguiente estructura:

```
\begin{multi}[opciones de pregunta]{nombre de pregunta}
texto de pregunta
\item[opciones]* cuestión correcta 1
\item[opciones] cuestión incorrecta 1
:
\item[opciones] cuestión incorrecta n
\end{multi}
```

Si la cuestión es correcta se designa mediante un asterisco (*) inmediatamente después del comando `\item`. El orden entre las respuestas es irrelevante.

Parámetros opcionales del comando `\item`

`fraction` Controla el peso de la respuesta. Por defecto, `\item*` asigna `fraction=100` y `\item` asigna `fraction=0`. Puede modificarse manualmente para otorgar puntuaciones parciales (por ejemplo: `\item[fraction=50]`).

`feedback` Define la retroalimentación que leerá el alumno tras contestar. Debe ir entre llaves. (por ejemplo: `\item[feedback={¡Muy bien!}]`).

```
\begin{multi}[points=3]{Superhéroes}
¿Quién es el superhéroe al que más le gusta el queso?
\item[feedback={Has respondido increíblemente mal}] Mister Increíble
\item[feedback={Muy bien. ¡Tómate un quesito de mi parte!}]* Super Ratón
\item[feedback={Tu respuesta es supermala}] Superman
\end{multi}
```

Parámetros opcionales del entorno `multi`

`multiple` Activado mediante `multiple` o `single=false`, permite ejercicios con varias cuestiones correctas. Admite dos modos de puntuación:

1. **Modo avanzado**: El profesor define las fracciones a mano (pueden ser negativas para penalizar).
2. **Modo automático**: Se marcan las correctas con `\item*` y el paquete distribuye el 100 % equitativamente entre ellas, aplicando penalizaciones automáticas a las incorrectas.

Por ejemplo, los siguientes dos ejemplos son equivalentes:

```
\begin{multi}[multiple]{Modo Automático}
¿Cuáles números son primos?
\item 2
\item* 5
\item* 7
\item 1
\item 6
\end{multi}
\begin{multi}[multiple]{Modo Avanzado}
¿Cuáles números son primos?
\item[fraction=-50] 2
\item[fraction=50] 5
\item[fraction=50] 7
\item[fraction=-50] 1
\item[fraction=-50] 6
\end{multi}
```

points Establece el valor de cada cuestión del cuestionario (por defecto `1`).

Nota sobre la puntuación en Moodle: A diferencia de AMC, donde se puede parametrizar una esperanza nula estricta con notas globales negativas, Moodle trunca la calificación mínima de una pregunta de opción múltiple a `0`. No permite notas globales negativas en estos entornos.

1.3.3. Escritura automatizada de las cuestiones desde Python

El formateo de los textos de aclaración y las respuestas correctas varía según si generamos el banco para los alumnos o la plantilla de revisión para el docente.

Para la versión del alumno, definimos `itemBuena` e `itemMala` encargadas de inyectar el parámetro `feedback`:

```
def itemBuena(aclaracion):
    return ("\\item[feedback={" + codchar(aclaracion) + "}]* ") if aclaracion else r"\item* "

def itemMala(aclaracion):
    return ("\\item[feedback={" + codchar(aclaracion) + "}]  ") if aclaracion else r"\item  "
```

Para la versión del profesor, estas funciones permiten visualizar el reparto de los puntos en función del número de cuestiones verdaderas y falsas (`numBuenas` o `numMalas`) para reflejar cómo penalizaría el formato en un diseño de examen estándar AMC:

```
def itemBuena(numBuenas, aclaracion):
    return ("\\item[fraction=" + str(round(100/numBuenas)) + feedback(aclaracion) + "]") if
    ↪  numBuenas else "\\item*"

def itemMala(numMalas, aclaracion):
    return ("\\item[fraction=" + str(-round(100/numMalas)) + feedback(aclaracion) + "]") if
    ↪  numMalas else "\\item"
```

La siguiente función auxiliar da formato al bloque de texto de la retroalimentación:

```
def feedback(texto):
    return r",\feedback={"+codchar(texto)+"}"
```

Procesamiento de las variantes extraídas del iterador Recordemos el desempaquetado de una variante generada por nuestro motor:

```
# 1. Instanciación del iterador
variantes = iter(ProblemaTipo(ejercicio))

# 2. Extracción y desempaquetado de una variante
id_pregunta, EnunciadoCompleto, lista_Cuestiones = next(variantes)
```

donde `lista_Cuestiones` es una lista de tuplas, donde cada tupla representa una de las opciones o subcuestiones a evaluar para esta variante. Por ejemplo `[('¿Se da D?', True, 1, 'Explicación para D'), ('¿Se dan C y D?', False, 1, '')]`

Cada tupla `c` de la `lista_Cuestiones` está estructurada de la siguiente forma:

```
('Texto de la opción/cuestión', True/False, 1/0, 'explicación si ha sido incluida')
```

y cada una es procesada en bucle según el destinatario:

Para la versión del estudiante Evaluamos la veracidad en `c[1]` para decidir el método de formateo aplicando una indentación limpia de 7 espacios.

```
for c in cuestiones:
    fb = c[3] if len(c) > 3 else ''
    s = s + (' ' * 7) + (itemBuena(fb) if c[1] else itemMala(fb)) + codchar(c[0]) + '\n'
```

Para la versión del profesor Controlamos además si la cuestión fue descartada por precondición (`c[2] == 0`), enviando el descarte al registro de diagnóstico global `ex`.

```
for c in cuestiones:
    if c[2]:
        s = s + (' ' * 7) + (itemBuena(b, c[3]) if c[1] else itemMala(m, c[3])) +
        ↪  codchar(c[0]) + "\n"
```

Gestión de la opción por defecto ("Ninguna de las anteriores") Moodle exige que exista al menos una opción correcta en el entorno o la compilación fallará. Para blindar el comportamiento si la combinatoria descarta todas las respuestas correctas de una variante, añadimos dinámicamente una opción por defecto cuya veracidad es complementaria al recuento de respuestas verdaderas.[1] El modo de hacerlo es añadir una tupla más en la lista `cuestiones`, que será falsa si en el resto de tuplas una o más cuestiones son verdaderas.

```
v = [c[1] for c in cuestiones].count(True)
cuestiones = cuestiones + [(codchar(lastchoice), (False if v else True), 1, '')]
```

1.3.4. `MoodleMulti`, `MoodleMultiProfe` y `MoodleMultiLastCh`

`MoodleMulti` Genera una cadena de caracteres con los comandos LaTeX correspondientes a una pregunta de opción múltiple dentro del entorno `{multi}`.

```
def MoodleMulti(nombre, variante, enunciado, cuestiones):
    <<Escritura del entorno multi>>
```

El método escribe el entorno aplicando los parámetros opcionales `multiple` y `points` (con valor igual al número de opciones de respuesta de la pregunta menos uno; es decir, la longitud de la lista `cuestiones` menos uno):

```
<<Definición itemBuena e itemMala para cuestiones en Moodle>>

s = " \\begin{multi}[multiple, points=" + str(len(cuestiones)-1) +"]"
s = s + "{" + codchar(nombre) + "-" + str(variante) + "}\n"
s = s + "    " + enunciado + "\n"

<<Escritura de las cuestiones para Moodle>>

s = s + " \\end{multi}\n\n"
return s
```

`MoodleMultiLastCh` Es similar a la función `MoodleMulti`, pero añadiendo la opción por defecto antes de generar el entorno.

```
def MoodleMultiLastCh(nombre, variante, enunciado, cuestiones,
                      lastchoice="Las demás opciones son falsas"):
    <<cuestion alternativa por defecto para Moodle>>
    <<Escritura del entorno multi>>
```

[1] Moodle no dispone de un equivalente a `\lastchoice` para fijar su posición al barajar, por lo que el texto recomendado es *"Las demás opciones son falsas"*.

`MoodleMultiProfe` Versión homóloga para la plantilla del docente. Realiza previamente el conteo de respuestas correctas (`b`) e incorrectas (`m`) para mostrar el porcentaje de puntuación de cada cuestión en el documento de revisión.

```
def MoodleMultiProfe(nombre, variante, enunciado, cuestiones):
    <<Escritura aclaración en Moodle>>
    <<Escritura del entorno multi para el Profe>>
```

Para calcular el valor para el argumento `fraction` del comando `\item`, necesitamos contar, entre las cuestiones no descartadas (`c[2]=1`), cuantas de cuestiones verdaderas (variable `b`) y cuántas son falsas (variable `m`).

```
b = 0; m = 0;
for c in cuestiones:
    if c[2]:
        if c[1]:
            b+=1
        else:
            m+=1
```

La estructura del código que escribe el entorno `{multi}` es similar al de las dos funciones anteriores:

```
<<Conteo de respuestas correctas e incorrectas>>
<<Definición itemBuena e itemMala para cuestiones en Moodle para Profe>>

s = " \\begin{multi}[multiple, fractiontol=5.1"  +"]"
s = s + "{" + codchar(nombre) + "-" + str(variante) + "}\n"
s = s + "    " + enunciado + "\n"

<<Escritura de las cuestiones para Moodle para el Profe>>
s = s + " \\end{multi}\n\n"
return s
```

1.3.5. `QuizMoodle`, `QuizAMCProfe` y `QuizVFMoodle`

Estas funciones de alto nivel abren el fichero con `with open()` y escriben en él las variantes. `QuizMoodle`, `QuizMoodleProfe` y `QuizVFMoodle` vuelcan antes el preámbulo generado por `_moodle_header` y cierran al final el entorno principal `quiz`; `QuizAMCProfe` escribe solo los bloques `\element` (sin preámbulo), que se incluyen en un documento AMC con `\input`.

`QuizMoodle`, `QuizMoodleProfe` y `QuizAMCProfe` aceptan como `problema` un objeto suelto o un diccionario `{nombre: problema}`; un auxiliar local (`creaDiccionario`) envuelve el objeto suelto en un diccionario de una entrada, de modo que el bucle interior es el mismo en ambos casos. `QuizVFMoodle` recibe en cambio un iterador de `ProblemaVF` (`GenVar`) y el número de variantes que se extraen de él.

`QuizMoodle` — función unificada `QuizMoodle` replica, en el contexto de Moodle, la misma estrategia de unificación que motivó `AMCblock` en AMC. Antes de esta versión, la opción de añadir la opción por defecto *«Las demás opciones son falsas»* requería una función completamente separada (`QuizMoodleLastCh`). Ahora, un único parámetro booleano controla ese comportamiento:

- `last_choice=False` (valor por defecto) → genera preguntas sin opción por defecto. Es el modo adecuado cuando la estructura del banco garantiza que siempre habrá al menos una respuesta correcta en cada variante.

- `last_choice=True` → inserta *«Las demás opciones son falsas»* como opción adicional, correcta si ninguna otra lo es y falsa en caso contrario. Es el modo habitual cuando el banco incluye variantes en que todas las respuestas podrían ser simultáneamente falsas.[2]

Internamente, la selección se implementa mediante una función de orden superior: la variable `cuerpo` apunta a `_ClozeMultiLastCh` o a `_ClozeMulti` según el valor de `last_choice`, y esa función se pasa como argumento a `_ClozeBlock` en el bucle de generación. Este patrón —almacenar en una variable la función que se quiere aplicar y pasarla como dato— evita un `if/else` repetido dentro del bucle de iteración y es una de las formas más limpias de escribir código condicional en Python.

`QuizMoodle` — función principal

```
def QuizMoodle(nombre, directorio, problema, last_choice=False,
               instances=1, *,
               aux_latex="", last_choice_text="Las demás opciones son falsas"):
    """Exporta un problema (o dict de problemas) al formato Moodle XML vía LaTeX.

    Parámetros
    ----------
    nombre           : nombre del quiz (identifica el grupo en Moodle)
    directorio       : ruta al directorio de salida (con o sin '/' final)
    problema         : ProblemaTipo o dict {nombre: ProblemaTipo}
    last_choice      : si True, añade la opción por defecto «las demás son falsas»
    aux_latex        : cabecera LaTeX adicional (p.ej. \\usepackage{...})
    last_choice_text : texto de la opción por defecto
    instances        : número de instancias con semillas distintas (ver por_partes())

    Retorna
    -------
    int : número de variantes escritas
    """
    def creaDiccionario(x, key='key'):
        return x if isinstance(x, dict) else {key: x}
    problema = creaDiccionario(problema, nombre)

    cuerpo = (lambda c: _ClozeMultiLastCh(c, last_choice_text)) if last_choice else _ClozeMulti

    count = 0
    with open(_os.path.join(directorio, nombre + ".tex"), "w", encoding="utf-8") as f:
        f.write(_moodle_header(nombre, auxLaTeX=aux_latex))
        for nom in problema:
            for etiqueta, partes in problema[nom].por_partes(instances=instances):
                f.write(_ClozeBlock(nom, etiqueta, partes, cuerpo))
                count += 1
        f.write("\\end{quiz}\n\n\\end{document}\n")
    return count
```

`QuizMoodleProfe` — versión docente Genera el banco para revisión del profesor, mostrando el reparto de puntos por cuestión. Acepta, solo por palabra clave, los mismos `last_choice`, `instances` y `last_choice_text` que `QuizMoodle`, de modo que muestra exactamente las variantes del alumno (con `last_choice=True` la opción por defecto se añade como una cuestión más, correcta si ninguna activa lo es, y entra en el reparto de fracciones):

[2] En Moodle, a diferencia de AMC, no existe un comando `\lastchoices` que fije la posición de esa opción al barajar. Por eso el texto recomendado es *«Las demás opciones son falsas»* en lugar de *«Ninguna de las anteriores»*: el primero es semánticamente correcto en cualquier posición de la lista.

```
def QuizMoodleProfe(nombre, directorio, problema, *, aux_latex="",
                    last_choice=False, instances=1,
                    last_choice_text="Las demás opciones son falsas"):
    """Vista profe para Moodle: mismas variantes que el alumno con fracciones visibles.

    Exporta las mismas variantes que QuizMoodle con los mismos last_choice e
    instances, pero con el esquema de corrección visible (porcentajes de
    puntuación por opción). Útil para que el profesor revise cómo puntúa
    Moodle cada variante.

    Para ver todas las cuestiones de cada hueco con sus marcas, usa
    QuizAMCProfe(), que genera un PDF AMC con \\explain{} para las rechazadas.

    Parámetros
    ----------
    nombre           : nombre del quiz (identifica el grupo en Moodle)
    directorio       : ruta al directorio de salida (con o sin '/' final)
    problema         : ProblemaTipo o dict {nombre: ProblemaTipo}
    aux_latex        : cabecera LaTeX adicional (p.ej. \\usepackage{...})
    last_choice      : si True, añade la opción por defecto (correcta solo si
                       las demás son falsas), como QuizMoodle(last_choice=True)
    instances        : número de instancias con semillas distintas (ver por_partes())
    last_choice_text : texto de la opción por defecto

    Retorna
    -------
    int : número de variantes escritas
    """
    def creaDiccionario(x, key='key'):
        return x if isinstance(x, dict) else {key: x}
    problema = creaDiccionario(problema, nombre)

    def cuerpo(cuestiones):
        if last_choice:
            ninguna = not any(c[2] and c[1] for c in cuestiones)
            cuestiones = cuestiones + [(last_choice_text, ninguna, 1, '')]
        return _ClozeMultiProfe(cuestiones)

    count = 0
    with open(_os.path.join(directorio, nombre + ".tex"), "w", encoding="utf-8") as f:
        f.write(_moodle_header(nombre, auxLaTeX=aux_latex))
        for nom in problema:
            for etiqueta, partes in problema[nom].por_partes(instances=instances):
                f.write(_ClozeBlock(nom, etiqueta, partes, cuerpo))
                count += 1
        f.write("\\end{quiz}\n\n\\end{document}\n")
    return count
```

`QuizAMCProfe`: vista profe en AMC `QuizMoodleProfe` muestra las mismas variantes que verá el alumno, con su puntuación. Para *auditar* una pregunta hace falta otra cosa: ver a la vez **todas** las cuestiones de cada hueco, con la marca de verdadera o falsa bajo cada mundo, y saber cuáles quedarían descartadas por su precondición. Eso es lo que produce `QuizAMCProfe`: recorre `por_partes_profe()` y escribe una pregunta AMC por mundo en `nombre_profe.tex` (`AMCblock(..., profe=True)` si tiene una parte, `AMC_multipart(..., profe=True)` si tiene varias; con `separadas=True`, en el modo de preguntas numeradas consecutivas), que se incluye en el documento AMC con `\input`. Las cuestiones rechazadas van al `\explain{}`. Devuelve el número

de entradas escritas (una por mundo, no por variante). Es la contrapartida en papel de la tabla de mundos de `qbank.lint`.

```
def QuizAMCProfe(nombre, directorio, problema, cols=1, *,
                 last_choice=False, aux_latex="",
                 last_choice_text="Ninguna de las anteriores",
                 separadas=False, ref_text=AMC_REF_TEXT):
    """Vista profe para AMC: todas las cuestiones de cada hueco.

    Usa por_partes_profe() para mostrar TODAS las cuestiones de cada hueco
    (no solo la seleccionada para el alumno) con marcas de correcto/incorrecto.
    Las cuestiones rechazadas por precondición aparecen en \\explain{}.

    Útil para revisar en papel la lógica de corrección de una pregunta.
    El fichero generado se incluye en el documento AMC con \\input{nombre_profe.tex}.

    Parámetros
    ----------
    nombre           : identificador del grupo AMC
    directorio       : ruta al directorio de salida (con o sin '/' final)
    problema         : ProblemaTipo o dict {nombre: ProblemaTipo}
    cols             : número de columnas multicols (1 = sin multicols)
    last_choice      : si True, añade la opción por defecto (como en la vista alumno)
    aux_latex        : instrucciones LaTeX extra dentro del bloque \\element{}
    last_choice_text : texto de la opción por defecto (solo con last_choice)
    separadas, ref_text : como en AMC_multipart (multiparte como preguntas
                      numeradas consecutivas)

    Retorna
    -------
    int : número de entradas escritas (un \\element por mundo)
    """
    def creaDiccionario(x, key='key'):
        return x if isinstance(x, dict) else {key: x}
    problema = creaDiccionario(problema, nombre)
    count = 0
    with open(_os.path.join(directorio, nombre + "_profe.tex"), "w", encoding="utf-8") as f:
        for nom in problema:
            for etiqueta, partes in problema[nom].por_partes_profe():
                if len(partes) > 1:
                    f.write(AMC_multipart(nom, etiqueta, "", partes,
                                          last_choice=last_choice, cols=cols, profe=True,
                                          aux_latex=aux_latex,
                                          last_choice_text=last_choice_text,
                                          separadas=separadas, ref_text=ref_text))
                else:
                    f.write(AMCblock(nom, etiqueta, partes[0][0], partes[0][1],
                                     last_choice=last_choice, cols=cols, profe=True,
                                     aux_latex=aux_latex,
                                     last_choice_text=last_choice_text))
                count += 1
    return count
```

`QuizLectura`: la vista de lectura La *vista de lectura* es un documento PDF para que el profesor **lea y opine** sobre las preguntas, distinto del catálogo AMC (que se queda para el examen): para cada pregunta, un ejemplo por mundo (y, con `setup`, por combinación de valores de las variables que la pregunta declare en `lectura.cubrir`), con el enunciado en un recuadro, las alternativas agrupadas por hueco, cada una marcada V (verde) o F (rojo) y con su `exp` interpolada debajo

en gris y cursiva, y las alternativas que la precondición excluye en ese mundo en gris más claro con su motivo. Al principio de cada pregunta va la tabla de mundos (`tabla_mundos`); al final, los mundos descartados por un `Supuesto` y las combinaciones de `cubrir` que no han aparecido.

Los ejemplos salen de `_ejemplos_lectura`, que recorre `por_partes_lectura()` y agrupa por `mundo`:

- Sin `setup`: un ejemplo por mundo.
- Con `setup` y sin `lectura.cubrir`: un ejemplo por mundo con la `seed` del problema y, con `extras=N`, hasta `N` sorteos más por mundo cuyo enunciado (sin cifras) sea distinto de los ya vistos.
- Con `lectura.cubrir`: recorre semillas (hasta `max_semillas`) y guarda, en cada mundo, el primer ejemplo de cada combinación de valores de las variables de `cubrir` (comparadas por `repr`). Como no se sabe de antemano qué combinaciones son posibles (una `precond` puede excluir algunas), se recorren todas las semillas; las combinaciones del producto cartesiano de los valores vistos que no hayan aparecido se listan en el documento como *no encontradas*. `MAX_EJEMPLOS_LECTURA` acota los ejemplos por pregunta.

Los ejemplos se ordenan por mundo y después por los valores de `cubrir`. `QuizLectura` acepta un `ProblemaTipo`, una lista de ellos (un documento por lección es el uso previsto) o un diccionario `{nombre: ProblemaTipo}`; con una lista, el nombre de cada pregunta es su `_nombre`. Escribe solo `directorio/nombre.tex` (un documento completo, en UTF-8) y devuelve el número de ejemplos; compilarlo queda para el `Makefile`. El preámbulo del proyecto (macros del curso, `babel`) llega por `aux_latex`, como en AMC y Moodle. `last_choice=None` significa leer `export.last_choice` de cada pregunta: es una excepción documentada a la regla de que los exportadores no leen `export`, porque en un mismo documento las preguntas difieren; la nota inicial de cada pregunta dice cuántas opciones ve el alumno y cuál es la opción por defecto. Los comandos del preámbulo llevan el prefijo `qb` (`\qbV`, `\qbF`, `\qbX`, `\qbExp`) para no chocar con macros de los proyectos.

```
MAX_EJEMPLOS_LECTURA = 40     # ejemplos por pregunta, como mucho

_CIFRAS_RE = _re.compile(r'-?\d+(?:[.,]\d+)?')


def _ejemplos_lectura(p, extras=0, max_semillas=400):
    """Ejemplos de la vista de lectura de un problema.

    Devuelve (ejemplos, descartes, faltan): ejemplos es una lista de
    (mundo, valores, registro Lectura) ordenada por mundo y valores, donde
    valores es la tupla de (variable, repr(valor)) de lectura.cubrir;
    descartes, los textos de los mundos que un Supuesto descarta (con la
    semilla del problema); faltan, las combinaciones de cubrir que no han
    aparecido en ningún mundo tras max_semillas semillas."""
    lectura = p.lectura if isinstance(p.lectura, dict) else {}
    cubrir  = [v for v in lectura.get('cubrir', []) if isinstance(v, str)]
    vistos, descartes = {}, []
    def clave(r):
        return tuple((v, _valor_lectura(r.ns.get(v))) for v in cubrir)
    def guarda(r, k):
        if (r.mundo, k) not in vistos and len(vistos) < MAX_EJEMPLOS_LECTURA:
            vistos[(r.mundo, k)] = r
            return True
        return False
    for r in p.por_partes_lectura(descartes=True):
        if r.descarte is not None:
```

```
            if r.descarte not in descartes:
                descartes.append(r.descarte)
        else:
            guarda(r, clave(r))
    if p.setup is not None and cubrir:
        base = p.seed if p.seed is not None else 0
        for semilla in range(base + 1, base + max_semillas):
            for r in p.por_partes_lectura(seed=semilla):
                guarda(r, clave(r))
    elif p.setup is not None and extras:
        base = p.seed if p.seed is not None else 0
        enunciados = {(r.mundo, tuple(_CIFRAS_RE.sub('#', pa.enunciado) for pa in r.partes))
                      for r in vistos.values()}
        cuantos = {m: 0 for m, _ in vistos}
        for semilla in range(base + 1, base + max_semillas):
            if all(n >= extras for n in cuantos.values()):
                break
            for r in p.por_partes_lectura(seed=semilla):
                e = (r.mundo, tuple(_CIFRAS_RE.sub('#', pa.enunciado) for pa in r.partes))
                if e in enunciados or cuantos.get(r.mundo, 0) >= extras:
                    continue
                enunciados.add(e)
                if guarda(r, (('sorteo', semilla),)):
                    cuantos[r.mundo] = cuantos.get(r.mundo, 0) + 1
    ejemplos = sorted(((m, k, r) for (m, k), r in vistos.items()),
                      key=lambda x: (x[0], tuple(_orden_lectura(val) for _, val in x[1])))
    faltan = []
    if cubrir:
        import itertools as _it
        mundos = {m for m, _ in vistos}
        valores = {v: sorted({dict(k)[v] for _, k in vistos}) for v in cubrir}
        for m in sorted(mundos):
            for combo in _it.product(*(valores[v] for v in cubrir)):
                k = tuple(zip(cubrir, combo))
                if (m, k) not in vistos:
                    faltan.append((m, k))
    return ejemplos, descartes, faltan


def _valor_lectura(x):
    """Valor de una variable del setup tal como se muestra y se compara:
    los escalares (también los de numpy) por su str(); el resto por repr()."""
    if isinstance(x, (str, bool)) or x is None:
        return str(x)
    try:
        return str(x.item()) if hasattr(x, 'item') else str(x)
    except Exception:                                       # noqa: BLE001
        return repr(x)


def _orden_lectura(x):
    """Clave de ordenación: numérica si el valor lo es, alfabética si no."""
    try:
        return (0, float(str(x).replace(',', '.')), '')
    except ValueError:
        return (1, 0.0, str(x))
```

```
def _tex_lectura_escapa(s):
    """Escapa en texto LaTeX los caracteres de un valor del setup."""
    return (str(s).replace('\\', r'\textbackslash{}').replace('_', r'\_').replace('%', r'\%')
            .replace('&', r'\&').replace('#', r'\#').replace('$', r'\$').replace('{', r'\{')
            .replace('}', r'\}').replace('~', r'\textasciitilde{}').replace('^', r'\^{}'))


def _fecha_lectura():
    import datetime as _dt
    hoy = _dt.date.today()
    meses = ('enero', 'febrero', 'marzo', 'abril', 'mayo', 'junio', 'julio', 'agosto',
             'septiembre', 'octubre', 'noviembre', 'diciembre')
    return f'{hoy.day} de {meses[hoy.month - 1]} de {hoy.year}'


def _tex_lectura_cabecera(titulo, aux_latex):
    return (
        '\\documentclass[11pt,a4paper]{article}\n'
        '\\usepackage[utf8]{inputenc}\n'
        '\\usepackage[T1]{fontenc}\n'
        '\\usepackage{amsmath,amssymb}\n'
        '\\usepackage[margin=2.2cm]{geometry}\n'
        '\\usepackage{xcolor,enumitem,graphicx}\n'
        + aux_latex.strip() + '\n'
        '\\definecolor{qbverde}{rgb}{0,0.5,0}\\definecolor{qbrojo}{rgb}{0.75,0,0}\n'
        '\\newcommand{\\qbV}{\\textbf{\\textcolor{qbverde}{V}}}\n'
        '\\newcommand{\\qbF}{\\textbf{\\textcolor{qbrojo}{F}}}\n'
        '\\newcommand{\\qbX}{\\textcolor{black!40}{--}}\n'
        '\\newcommand{\\qbExp}[1]{\\par\\vspace{1pt}{\\small\\color{black!65}\\emph{#1}}\\par}
        ↪  \n'
        '\\newcommand{\\qbRecuadro}[1]{\\noindent\\fbox{\\parbox{\\dimexpr\\linewidth-2\\fboxs
        ↪  ep-2\\fboxrule}{#1}}}\n'
        '\\setlength{\\parindent}{0pt}\\setlength{\\parskip}{4pt}\n'
        '\\title{' + titulo + '}\\date{' + _fecha_lectura() + '}\n'
        '\\begin{document}\\maketitle\n'
        'Un ejemplo por cada mundo de cada pregunta (y, si la pregunta lo pide, por cada '
        'combinación de valores de su \\texttt{setup}). En cada hueco el alumno recibe '
        '\\emph{una} de las alternativas listadas. \\qbV/\\qbF{} es el valor de verdad en ese '
        'ejemplo; debajo, en gris, la retroalimentación. Las alternativas que la precondición '
        'excluye en ese mundo van en gris claro, con su motivo.\n\n')


def _tex_lectura_pregunta(nombre, p, ejemplos, descartes, faltan, last_choice,
↪  last_choice_text):
    from qbank._lint import tabla_mundos
    L = ['\\clearpage\\section*{' + _tex_lectura_escapa(nombre) + '}\n']
    if ejemplos:
        n_huecos = [len(pa.huecos) for pa in ejemplos[0][2].partes]
        opciones = [n + (1 if last_choice else 0) for n in n_huecos]
        def plural(o):
            return f'{o} opción' if o == 1 else f'{o} opciones'
        if len(opciones) == 1:
            nota = 'El alumno ve ' + plural(opciones[0])
        else:
            nota = 'El alumno ve ' + ', '.join(f'{plural(o)} en la parte {k + 1}'
                                               for k, o in enumerate(opciones))
        if last_choice:
            nota += ('; la última' if len(opciones) > 1 or opciones[0] != n_huecos[0] + 1
```

```
                else f'; la opción {opciones[0]}') + \
               ' es «' + last_choice_text + '», correcta solo si las demás son falsas'
    L.append(nota + '.\n\n')
try:
    # 96 caracteres: los que caben en \\footnotesize monoespaciada con margin=2.2cm.
    # El cierre va partido: entero, cerraría el verbatim del PDF de este .org.
    L.append('{\\footnotesize\\begin{verbatim}\n' + tabla_mundos(p, linea=96,
    ↪  elipsis='...')
          + '\n\\end' '{verbatim}}\n')
except Exception as e:                                  # noqa: BLE001
    L.append('{\\small\\color{black!65}(sin tabla de mundos: ' + _tex_lectura_escapa(e) +
    ↪  ')}\n\n')
# Solo los slots de enunciado con más de una opción identifican el mundo.
multi = [j for j, x in enumerate(x for x in p.e
                                 if _clasifica_slot(_normaliza_slot(x)) != 'cuestiones')
         if len(_normaliza_slot(x)) > 1]
def nombre_mundo(mundo):
    return ('Mundo ' + '.'.join(str(mundo[j] + 1) for j in multi)) if multi else 'Mundo
    ↪  único'
for mundo, valores, r in ejemplos:
    cab = nombre_mundo(mundo)
    if valores:
        cab += ' · ' + ' · '.join(f'{_tex_lectura_escapa(v)} = {_tex_lectura_escapa(x)}'
                                  for v, x in valores)
    L.append('\\subsection*{' + cab + '}\n')
    h = 0
    for k, pa in enumerate(r.partes):
        if pa.enunciado.strip():
            L.append('\\qbRecuadro{' + pa.enunciado.strip() + '}\\par\\medskip\n')
        for hueco in pa.huecos:
            h += 1
            L.append('\\par\\textbf{Hueco ' + str(h) +
            ↪  '}\\begin{itemize}[leftmargin=2.2em,itemsep=3pt]\n')
            prev = None
            for a in hueco:
                if a.valor is None:
                    L.append('\\item[\\qbX] {\\color{black!40} ' + a.texto + ' --- '
                             + _tex_lectura_escapa(a.motivo) + '}\n')
                    continue
                exp = ('la misma retroalimentación que la anterior'
                       if a.exp and a.exp == prev else a.exp)
                L.append('\\item[' + ('\\qbV' if a.valor else '\\qbF') + '] ' + a.texto
                         + ('\\qbExp{' + exp + '}' if exp else '') + '\n')
                prev = a.exp
            L.append('\\end{itemize}\n')
if descartes:
    L.append('\\par{\\small\\color{black!65}Mundos descartados: '
             + '; '.join(_tex_lectura_escapa(d) for d in descartes) + '.}\n')
if faltan:
    L.append('\\par{\\small\\color{black!65}Combinaciones no encontradas: '
             + '; '.join(nombre_mundo(m) + ' con '
                         + ', '.join(f'{_tex_lectura_escapa(v)} = {_tex_lectura_escapa(x)}'
                                     for v, x in k) for m, k in faltan) + '.}\n')
if not ejemplos:
    L.append('{\\color{qbrojo}Sin ninguna variante válida.}\n')
return ''.join(L)
```

```
def QuizLectura(nombre, directorio, problemas, *, aux_latex="", titulo=None,
                last_choice=None, last_choice_text="Ninguna de las anteriores",
                extras=0, max_semillas=400):
    """Vista de lectura: escribe directorio/nombre.tex, un documento LaTeX
    completo con, por cada pregunta, un ejemplo por mundo (y por combinación
    de valores de lectura.cubrir), las alternativas por hueco con V/F y su
    exp, y las excluidas por precondición con su motivo.

    problemas        : ProblemaTipo, lista de ProblemaTipo (nombre = _nombre)
                       o dict {nombre: ProblemaTipo}
    aux_latex        : preámbulo del proyecto (macros, babel), como en AMC/Moodle
    titulo           : título del documento (por defecto, nombre)
    last_choice      : None lee export.last_choice de cada pregunta; True/False
                       lo fija para todas (solo afecta a la nota inicial)
    last_choice_text : texto de la opción por defecto en esa nota
    extras           : sin lectura.cubrir, sorteos extra por mundo con enunciado
                       distinto (solo con setup)
    max_semillas     : semillas que se recorren buscando combinaciones

    Devuelve el número de ejemplos escritos."""
    if isinstance(problemas, dict):
        pares = list(problemas.items())
    elif isinstance(problemas, (list, tuple)):
        pares = [(getattr(p, '_nombre', '') or f'pregunta{i + 1}', p) for i, p in
        ↪  enumerate(problemas)]
    else:
        pares = [(nombre, problemas)]
    total = 0
    partes = [_tex_lectura_cabecera(titulo if titulo is not None else
    ↪  _tex_lectura_escapa(nombre),
                                    aux_latex)]
    for nom, p in pares:
        ejemplos, descartes, faltan = _ejemplos_lectura(p, extras=extras,
        ↪  max_semillas=max_semillas)
        lc = (p.export.get('last_choice', False) if isinstance(p.export, dict) else False) \
             if last_choice is None else last_choice
        partes.append(_tex_lectura_pregunta(nom, p, ejemplos, descartes, faltan, lc,
        ↪  last_choice_text))
        total += len(ejemplos)
    partes.append('\\end{document}\n')
    with open(_os.path.join(directorio, nombre + '.tex'), 'w', encoding='utf-8') as f:
        f.write(''.join(partes))
    return total
```

QuizVFMoodle Para los bancos de preguntas `ProblemaVF`, las funciones homólogas extraen variantes basadas en un iterador clásico mediante el método `'next()'`, permitiendo fijar el número exacto de preguntas deseadas:

```
def QuizVFMoodle(nombre, directorio, GenVar, num, opc=None):
    if opc is None:
        opc = ["", ""]
    auxLaTeX = opc[0]
    with open(_os.path.join(directorio, nombre + ".tex"), "w", encoding="utf-8") as f:
        f.write(_moodle_header(nombre, auxLaTeX=auxLaTeX))
        for i in range(num):
            var = next(GenVar)
```

```
            f.write(MoodleMulti(codchar(nombre), var[0], codchar(var[1]), var[2]))
        f.write("\\end{quiz}\n\n\\end{document}\n")
```

1.3.6. Ejemplo de uso de `QuizMoodle`

A continuación se ilustra la ejecución de scripts independientes para generar tanto un bloque relacional condicionado (`ProblemaTipo`) como una batería aleatoria estructurada desde una lista estática de enunciados (`ProblemaVF`):

```
from qbank import *
p = ProblemaTipo([
      "Considere ",
      [
          Supuesto("$\\mathcal{A}$ ", v("A")),
          Supuesto("$\\mathcal{B}$ ", v("B")),
      ],
      [
          Supuesto("y que $\\mathcal{A}\\Rightarrow\\mathcal{C}$. ", v("A") >> v("C")),
          Supuesto("y que $\\mathcal{B}\\Leftrightarrow\\mathcal{D}$. ", v("B") ** v("D")),
      ],
      "Indique qué opciones son correctas: ",
      Cuestion("Entonces $\\mathcal{C}$ es verdadero", v("C")),
      Cuestion("Entonces $\\mathcal{D}$ es verdadero", v("D")),
      Cuestion("Entonces $\\mathcal{E}$ es verdadero", v("E")),
])
import os
nombre = "EjemploMoodle"
directorio = "ejemplos/"
os.makedirs(directorio, exist_ok=True)
QuizMoodle(nombre, directorio, p, last_choice=True)

from qbank import *

enunciado = "Indique qué afirmaciones son verdaderas:"
banco = [
  ("Todo alumno que va a clase aprueba",   False),
  ("Todo alumno que suspende va a clase",  False),
  ("Todo alumno que sabe aprueba",         True),
  ("Si aprueban todos eres buen profesor", False),
  ("Si suspenden todos eres mal profesor", False),
  ("Si suspenden todos es frustrante",     True),
]

b = [(codchar(p[0]), p[1]) for p in banco]
GenVar = iter(ProblemaVF(codchar(enunciado), b, 3))

import os
nombre = "EjemploVFMoodle"
directorio = "ejemplos/"
os.makedirs(directorio, exist_ok=True)
QuizVFMoodle(nombre, directorio, GenVar, 4)
```

1.4. Preguntas multi-parte (AMC) y cloze (Moodle)

Los formatos AMC y Moodle permiten agrupar varias sub-preguntas de opción múltiple bajo un único enunciado general. Cada sub-pregunta tiene su propio texto introductorio y su propio bloque de opciones.

1.4.1. `AMC_multipart` — Bloque AMC multi-parte

Genera un único bloque `\begin{questionmult}` (un solo identificador por variante) que contiene un grupo de casillas por parte, cada uno envuelto en `\AMCnoCompleteMulti`. Recibe directamente la lista `partes` que devuelven `por_partes()` y `por_partes_profe()`: el enunciado de la primera parte (que contiene las hipótesis) se imprime tal cual y los de las siguientes en cursiva, como introducción de su bloque. Admite los mismos parámetros que `AMCblock` y los aplica a cada parte, de modo que con `last_choice` cada bloque lleva su propia opción por defecto y la garantía de mezcla se conserva parte a parte. Es lo que deben usar los scripts de exportación cuando `len(partes) > 1`: escribir un `AMCblock` por parte produce dos preguntas con el mismo identificador y la segunda sin las hipótesis.

```
AMC_REF_TEXT = "Las preguntas {a} a {b} se refieren al mismo enunciado."


def AMC_multipart(nombre, etiqueta, enunciado, partes,
                  last_choice=False, cols=1, profe=False, *,
                  aux_latex="", last_choice_text="Ninguna de las anteriores", opc=None,
                  separadas=False, ref_text=AMC_REF_TEXT):
    """Bloque AMC de una pregunta multiparte: un único questionmult con un
    bloque de casillas por parte o, con separadas=True, un questionmult por
    parte dentro del mismo \\element (preguntas numeradas consecutivas).

    partes    : [(enunciado_parte, cuestiones), ...] tal como lo devuelven
                por_partes() y por_partes_profe()
    enunciado : texto común que precede a todas las partes; normalmente ""
                porque el enunciado de la primera parte ya lo contiene
    last_choice, cols, profe, aux_latex, last_choice_text : como en AMCblock,
                aplicados a cada parte (con last_choice, cada parte lleva su
                propia opción por defecto)
    opc       : forma antigua de pasar aux_latex ([aux_latex]); se mantiene
                por compatibilidad
    separadas : si True, una pregunta AMC por parte (identificadores
                nombre-etiqueta-pK), precedidas de ref_text
    ref_text  : frase previa del modo separado; {a} y {b} se sustituyen por
                los números de la primera y la última pregunta (\\AMCref)
    """
    if opc:
        aux_latex = opc[0]
    if separadas and len(partes) > 1:
        return _amc_partes_separadas(nombre, etiqueta, enunciado, partes, last_choice,
                                     cols, profe, aux_latex, last_choice_text, ref_text)
    nombre_tex = _sanitize_name(nombre)
    s  = '\\element{' + nombre_tex + '}{' + aux_latex + '\n'
    s += ' \\begin{questionmult}{' + nombre_tex + '-' + str(etiqueta) + '}\n'
    if enunciado:
        s += '  ' + _fix_display(enunciado) + '\n\n'
    ex = ''
    for k, (intro, cuestiones) in enumerate(partes):
        if intro.strip():
            intro_tex = _fix_display(intro.strip())
            s += '  ' + (intro_tex if k == 0 and not enunciado else '\\emph{' + intro_tex +
            ↪  '}') + '\n'
        choices, ex_parte = _amc_choices(cuestiones, last_choice, cols, profe,
        ↪  last_choice_text)
        s += '  {\\AMCnoCompleteMulti\n' + choices + '  }\n\n'
        ex += ex_parte.replace(' cuestión:', f' parte {k + 1}, cuestión:')
```

```
    if profe and ex:
        s += '   \\explain{' + ex + '            }\n'
    s += ' \\end{questionmult} '
    s += '}\n\n'
    return s


def _amc_partes_separadas(nombre, etiqueta, enunciado, partes, last_choice, cols,
                          profe, aux_latex, last_choice_text, ref_text):
    """Modo separado de AMC_multipart: un questionmult por parte en un solo
    \\element, con la frase «las preguntas N a M…» delante."""
    nombre_tex = _sanitize_name(nombre)
    ids = [f'{nombre_tex}-{etiqueta}-p{k + 1}' for k in range(len(partes))]
    frase = (ref_text.replace('{a}', '\\AMCref{' + ids[0] + '}')
                     .replace('{b}', '\\AMCref{' + ids[-1] + '}'))
    # El \\par final separa la frase del \\AMCbeginQuestion de la primera
    # pregunta (si no, su \\hspace negativo se pega a la frase).
    s  = '\\element{' + nombre_tex + '}{' + aux_latex + '\n'
    s += ' \\par\\vspace{1ex}\\noindent\\textbf{' + frase + '}\\par\n'
    for k, (intro, cuestiones) in enumerate(partes):
        etiquetada = k in (0, len(partes) - 1)
        s += (' \\begin{questionmult}{' + ids[k] + '}'
              + ('\\AMCqlabel{' + ids[k] + '}' if etiquetada else '') + '\n')
        if k == 0 and enunciado:
            s += '  ' + _fix_display(enunciado) + '\n\n'
        if intro.strip():
            s += '  ' + _fix_display(intro.strip()) + '\n'
        choices, ex = _amc_choices(cuestiones, last_choice, cols, profe, last_choice_text)
        s += '  {\\AMCnoCompleteMulti\n' + choices + '  }\n'
        if profe and ex:
            s += '   \\explain{' + ex + '            }\n'
        s += ' \\end{questionmult}\n'
    s += '}\n\n'
    return s
```

Con `separadas=True` cada parte es una pregunta AMC propia (`\begin{questionmult}{nombre-etiqueta-pK}`), todas dentro del mismo `\element`: como `\shufflegroup` y `\copygroup` mueven elementos enteros, las partes viajan juntas y en orden y se numeran consecutivamente. Cada una tiene su fila en la hoja de respuestas y puntúa como una pregunta normal; un bloque de k partes cuenta, pues, como k preguntas del examen (aunque `\copygroup[4]{…}` sigue escogiendo cuatro *elementos*). Delante va `ref_text` (por defecto `AMC_REF_TEXT`, «Las preguntas {a} a {b} se refieren al mismo enunciado.»), en negrita y terminada en `\par`; `{a}` y `{b}` se sustituyen por `\AMCref` a las etiquetas que `\AMCqlabel` pone en la primera y la última pregunta. AMC prefija esas etiquetas con el número de copia, así que el mismo nombre vale en todas las copias; los números reales aparecen tras la segunda pasada de LaTeX. Las intros de las partes no van en cursiva (son el enunciado de su pregunta) y, con `profe=True`, cada pregunta lleva su propio `\explain`. Con una sola parte no hay frase ni etiquetas: sale lo mismo que en el modo por defecto.

Se usa en un bucle sobre `por_partes()`, pasando la lista de partes entera (el enunciado común va vacío: cada parte lleva el suyo):

```
with open("preguntas.tex", "w") as f:
    for etiqueta, partes in p.por_partes():
        f.write(AMC_multipart("MiCuestionario", etiqueta, "", partes, last_choice=True))
```

1.4.2. Bloques cloze: `_ClozeMulti`, `_ClozeMultiProfe`, `_ClozeMultiLastCh` y `_ClozeBlock`

Toda la exportación a Moodle usa el entorno *cloze*, tenga el ejercicio una o varias partes. Cada variante produce una pregunta `\begin{cloze}{nombre-etiqueta}` que contiene un entorno `\begin{multi}` por cada parte (sin nombre, como exige el paquete `moodle` para los entornos embebidos). El feedback por opción, las fracciones del modo profe y el last-choice funcionan dentro de cloze (manual de `moodle.sty` §2.4.7 y documentación de Moodle), por lo que reutilizamos exactamente la misma lógica de escritura de `\item` que las preguntas sueltas.

`_ClozeMulti`, `_ClozeMultiProfe` y `_ClozeMultiLastCh` generan el bloque `\begin{multi}…\end{multi}` de **una** parte (versión alumno, profe y last-choice respectivamente); `_ClozeBlock` envuelve las partes de una variante en el entorno `\begin{cloze}`.

El identificador del entorno cloze se sanitiza mediante `_sanitize_name` (sustituye `:` y `_` por `-`) y se codifica con `codchar` para compatibilidad con LaTeX sin Unicode nativo.

```
def _ClozeMulti(cuestiones):
    def itemBuena(aclaracion):
        return ("\\item[feedback={" + codchar(aclaracion) + "}]* ") if aclaracion else
        ↪  r"\item* "

    def itemMala(aclaracion):
        return ("\\item[feedback={" + codchar(aclaracion) + "}]  ") if aclaracion else r"\item
        ↪   "

    s = "  \\begin{multi}[multiple, points=" + str(len(cuestiones)-1) + "]\n"
    <<Escritura de las cuestiones para Moodle>>
    s = s + "  \\end{multi}\n\n"
    return s

def _ClozeMultiLastCh(cuestiones, lastchoice="Las demás opciones son falsas"):
    <<cuestion alternativa por defecto para Moodle>>
    def itemBuena(aclaracion):
        return ("\\item[feedback={" + codchar(aclaracion) + "}]* ") if aclaracion else
        ↪  r"\item* "

    def itemMala(aclaracion):
        return ("\\item[feedback={" + codchar(aclaracion) + "}]  ") if aclaracion else r"\item
        ↪   "

    s = "  \\begin{multi}[multiple, points=" + str(len(cuestiones)-1) + "]\n"
    <<Escritura de las cuestiones para Moodle>>
    s = s + "  \\end{multi}\n\n"
    return s

def _ClozeMultiProfe(cuestiones):
    <<Escritura aclaración en Moodle>>
    <<Conteo de respuestas correctas e incorrectas>>
    <<Definición itemBuena e itemMala para cuestiones en Moodle para Profe>>

    s = "  \\begin{multi}[multiple, fractiontol=5.1]\n"
    for c in cuestiones:
        if c[2]:
            s = s + (' ' * 7) + (itemBuena(b, c[3]) if c[1] else itemMala(m, c[3])) +
            ↪  codchar(c[0]) + "\n"
    s = s + "  \\end{multi}\n\n"
    return s

def _ClozeBlock(nombre, etiqueta, partes, cuerpo):
```

```
    """Envuelve las partes de una variante en un entorno cloze.
    `cuerpo(cuestiones)` genera el bloque \\begin{multi}…\\end{multi} de cada parte."""
    nombre_tex = codchar(_sanitize_name(nombre))
    s = " \\begin{cloze}{" + nombre_tex + "-" + str(etiqueta) + "}\n"
    for enunciado, cuestiones in partes:
        s += "  " + codchar(enunciado) + "\n"
        s += cuerpo(cuestiones)
    s += " \\end{cloze}\n\n"
    return s
```

2. Plantilla de proyecto: `qbank-init`

Un banco de preguntas real no es solo un conjunto de JSON: necesita los scripts que los recorren y exportan, un `Makefile` que compile los PDF y los XML, y un sitio por donde empezar. El módulo `qbank._plantilla` distribuye con el paquete una plantilla de proyecto (`src/qbank/plantilla/`) y la orden `qbank-init DIR` que la copia:

```
qbank-init mi-banco
cd mi-banco
make lint          # comprueba las preguntas de ejemplo
make amc           # PDF de revisión (vista profe) en build/amc/
```

La plantilla contiene `preguntas/` (cinco preguntas de ejemplo que cubren mundos alternativos, axiomas, `precond`, `setup` con `seed` e `instances`, multiparte y la pareja de afirmaciones verdadera/falsa), `scripts/` (`gen_amc.py`, `gen_amc_student.py`, `gen_moodle.py` y auxiliares), `Makefile` (`lint`, `amc`, `amc-student`, `moodle`, filtro `LECCION`), `config.py` (preámbulo LaTeX y textos de la opción por defecto), `nueva_pregunta.py` (escribir una pregunta en Python y guardarla como JSON) y `README.org` con el flujo de trabajo. Al vivir dentro del paquete, la plantilla que recibe cada versión de `qbank` es la que funciona con esa versión, y `tests/test_plantilla.py` ejecuta sus scripts sobre sus preguntas de ejemplo en cada cambio.

```
# Copyright (C) 2020-2026  Marcos Bujosa
#
# This file is part of calcprop-qbank.
#
# calcprop-qbank is free software: you can redistribute it and/or modify
# it under the terms of the GNU General Public License as published by
# the Free Software Foundation, either version 3 of the License, or
# (at your option) any later version.
#
# calcprop-qbank is distributed in the hope that it will be useful,
# but WITHOUT ANY WARRANTY; without even the implied warranty of
# MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
# GNU General Public License for more details.
#
# You should have received a copy of the GNU General Public License
# along with this program.  If not, see <https://www.gnu.org/licenses/>.

"""Plantilla de proyecto para un banco de preguntas (``qbank-init``).

    qbank-init mi-banco                # crea mi-banco/ a partir de la plantilla
    python -m qbank._plantilla mi-banco # equivalente

La plantilla vive en ``qbank/plantilla/`` dentro del paquete: preguntas de
ejemplo, scripts de generación AMC/Moodle, Makefile, un guion para escribir
preguntas en Python (``nueva_pregunta.py``) y un README con el flujo de
```

```
trabajo. Tras crear el proyecto: ``cd mi-banco && make lint && make amc``.
"""
import argparse as _argparse
import shutil as _shutil
import sys as _sys
from importlib import resources as _resources
from pathlib import Path as _Path

__all__ = ['ruta_plantilla', 'crear', 'main']

_IGNORAR = _shutil.ignore_patterns('__pycache__', '*.pyc', 'build')


def ruta_plantilla():
    """Directorio de la plantilla dentro del paquete instalado."""
    return _Path(str(_resources.files('qbank').joinpath('plantilla')))


def crear(destino, forzar=False):
    """Copia la plantilla en ``destino`` y devuelve su ruta.

    Se niega a escribir en un directorio no vacío salvo con ``forzar=True``
    (entonces sobreescribe los ficheros de la plantilla y conserva el resto).
    """
    destino = _Path(destino)
    if destino.exists() and any(destino.iterdir()) and not forzar:
        raise FileExistsError(f"{destino} existe y no está vacío (usa --forzar para
        ↪  sobreescribir)")
    _shutil.copytree(ruta_plantilla(), destino, ignore=_IGNORAR, dirs_exist_ok=True)
    return destino


def main(argv=None):
    parser = _argparse.ArgumentParser(
        prog='qbank-init',
        description='Crea un proyecto de banco de preguntas qbank a partir de la plantilla.')
    parser.add_argument('destino', help='carpeta del nuevo proyecto (se crea si no existe)')
    parser.add_argument('--forzar', action='store_true',
                        help='escribir aunque la carpeta no esté vacía (sobreescribe la
                        ↪  plantilla)')
    args = parser.parse_args(argv)
    try:
        destino = crear(args.destino, args.forzar)
    except OSError as e:                    # existe y no está vacío, es un fichero, sin
    ↪  permisos…
        print(f"qbank-init: {e}", file=_sys.stderr)
        return 1
    print(f"Proyecto creado en {destino}/\n"
          f"\n  cd {destino}\n"
          f"  make lint        # comprueba las preguntas de ejemplo\n"
          f"  make amc         # PDF de revisión (vista profe) en build/amc/\n"
          f"\nEl flujo completo está en {destino}/README.org")
    return 0


if __name__ == '__main__':
    _sys.exit(main())
```